Abstract

Rationale: Imaging data driven artificial intelligence research in Nuclear Medicine increasingly depends on large-scale multi-modal 3D/4D datasets (PET/SPECT with CT/MRI) that must be resampled, aligned and quantitatively normalized without losing the spatial and clinical metadata required for physical interpretation (e.g. SUV, absorbed dose). Existing pipelines often struggle with (1) biobank-scale volume and I/O overhead, (2) execution speed and the "two-language" barrier, (3) metadata drift when images are converted to raw arrays (4) and end-to-end differentiability for novel physics-informed machine learning.

Methods: We developed `MedImages.jl`, a native Julia library that binds voxel data to spatial metadata (origin, spacing, direction) and clinical/temporal fields using a typed `MedImage` structure and a batched multimodal container (`BatchedMedImage`) for synchronized transforms. Volume-scale performance was evaluated on a 100-subject autoPET PET/CT alignment pipeline (target grid 512^3) comparing HDF5-based persistence to a Python caching pipeline. The execution speed was benchmarked on GPU for resampling, orientation changes, and fused affine transforms, and compared with SimpleITK (ITK-based C++ reference toolkit). Differentiability was validated via gradient tests and learned spatial pipelines (inverse-rotation spatial transformer; organ-specific affine registration) and extended to physics-informed dosimetry using a Universal Differential Equation (UDE) for ^{177}Lu-PSMA trained against Monte Carlo dose ground truths. Metadata integrity was assessed by SUV agreement with SimpleITK and by an end-to-end PET/CT transformation chain with lesion-based readouts.

Results: `MedImages.jl` achieved a 7.2× turnaround speedup versus caching in the 100-subject benchmark and up to 115× (resampling), 71× (orientation), and 135× (fused affine) GPU speedups versus CPU. SUVbw matched SimpleITK to 10^{-14} (double precision), and compounded transforms preserved Mean SUV (3.4913 → 3.4660; 0.73%) and lesion volume (52.55 → 51.86 cm^3; 1.32%), consistent with interpolation effects rather than metadata deterioration. Learned spatial transforms reduced reconstruction error by $> 65\%$, and UDE dosimetry refinement converged to MSE 4.14×10^{-4} versus Monte Carlo ground-truth dose maps.

Conclusion: `MedImages.jl` provides a unified, GPU-accelerated, metadata-aware, and differentiable foundation that addresses scalability, speed, differentiability, and metadata integrity for quantitative nuclear medicine preprocessing and AI/SciML workflows.

Key words: nuclear medicine; PET/CT; SUV; metadata; differentiable programming

1. Introduction

Nuclear medicine diagnostic data, such as PET and SPECT, provide quantitative measurements of biologic processes that support oncologic staging, assessment of therapy response, and planning of theranostic treatment. Correct processing and interpretation of functional imaging data require that measured uptake is assigned to the same patient-space coordinates as the anatomic reference (CT or MRI). Hybrid imaging therefore combines complementary modalities within a single, physically defined sampling space specified by image origin, grid or voxel spacing, and orientation.

As multimodal imaging is increasingly reused as input for AI models, managing data volume and maintaining heterogeneous acquisition metadata has become a central practical challenge []. A high computational data preparation process typically requires: loading of a large number of examinations from disk, mapping each modality into a shared patient-space coordinate system, resampling onto standardized grids and applying geometric transforms for augmentation or registration. Preserving spatial and quantitative metadata is necessary throughout the multistep preprocessing phase so that voxel values remain physically interpretable.

Metadata management is not only a practical but also a safety-relevant issue. Converting medical image objects (DICOM, NIfTI) to raw arrays for deep learning can decouple voxel intensities from their patient-space definition and from clinical metadata used for quantification. This "metadata drift" can yield visually plausible results that are spatially inconsistent or quantitatively invalid [].

In widely used image-processing environments such as SimpleITK [**?**] and MONAI [**?**], data handling is often built around caching workflows and chains of separate libraries rather than architectures tailored to biobank-scale diagnostic imaging. As a result, a substantial fraction of total processing time is used not only for core preprocessing operations itself, but also to repeatedly copy very large image volumes between intermediate storage steps and different software components, and to ensure the consistency of spatial information [**?**].

.

These limitations become even more pronounced in emerging Scientific Machine Learning (SciML) workflows, which increasingly rely on end-to-end optimization in which geometric preprocessing participates in training of AI models. In nuclear medicine, such workflows include learned registration and motion correction, data augmentation, and physics-informed models for image reconstruction or dosimetry. Although Python-based frameworks provide differentiable operations, practical composability across heterogeneous software components often requires committing the workflow to a single tensor ecosystem, creating friction when integrating third-party tools or classical numerical solvers [**?** **?**].

To address these limitations, we developed MedImages.jl, a Julia-based imaging ecosystem that converts high-level code into efficient machine instructions using just-in-time intermediate optimization and enables native implementations of geometric image operations. As a result, these operations can run efficiently on both CPUs and GPUs without calling separate compiled libraries [**?**].

MedImages.jl binds voxel data to spatial and acquisition metadata through typed image representations, so that the typical for preprocessing geometric operations update the underlying physical description rather than operating on detached arrays. Core operators like resampling, affine warping, orientation handling, cropping and padding are implemented as native Julia kernels with transparent GPU acceleration, reducing practical bottlenecks in repeated 3D processing, supporting multimodal synchronization through batched representations.

The new framework is designed for interoperability with the broader Julia SciML ecosystem and with automatic differentiation libraries such as Zygote.jl and Enzyme.jl, which allows for differentiable spatial pipelines [**?**]. This permits geometric image operations to be incorporated directly into end-to-

end optimization, allowing spatial transformations to influence model training rather than remain external preprocessing steps. Julia further reduces software fragmentation by enabling image transformations, reconstruction algorithms, numerical solvers, and optimization routines to operate within a single coherent framework instead of across separate software layers that must be manually linked [**?**]. Since these compiler-level differentiation engines can act on arbitrary Julia code, gradients can propagate through solvers and GPU kernels without rewriting the underlying workflow in a specialized tensor framework [**?**].

In this work, we evaluate MedImages.jl across nuclear medicine–relevant benchmarks and experiments, both in isolation and in integration with complementary Julia libraries, with emphasis on (i) scalability for large PET/CT cohorts with high preprocessing demands, (ii) execution speed without the maintenance costs of a two-language architecture, (iii) metadata preservation to maintain the physical interpretability required for quantitative nuclear medicine, and (iv) differentiability of geometric operations for learned spatial transforms.

2. Methods

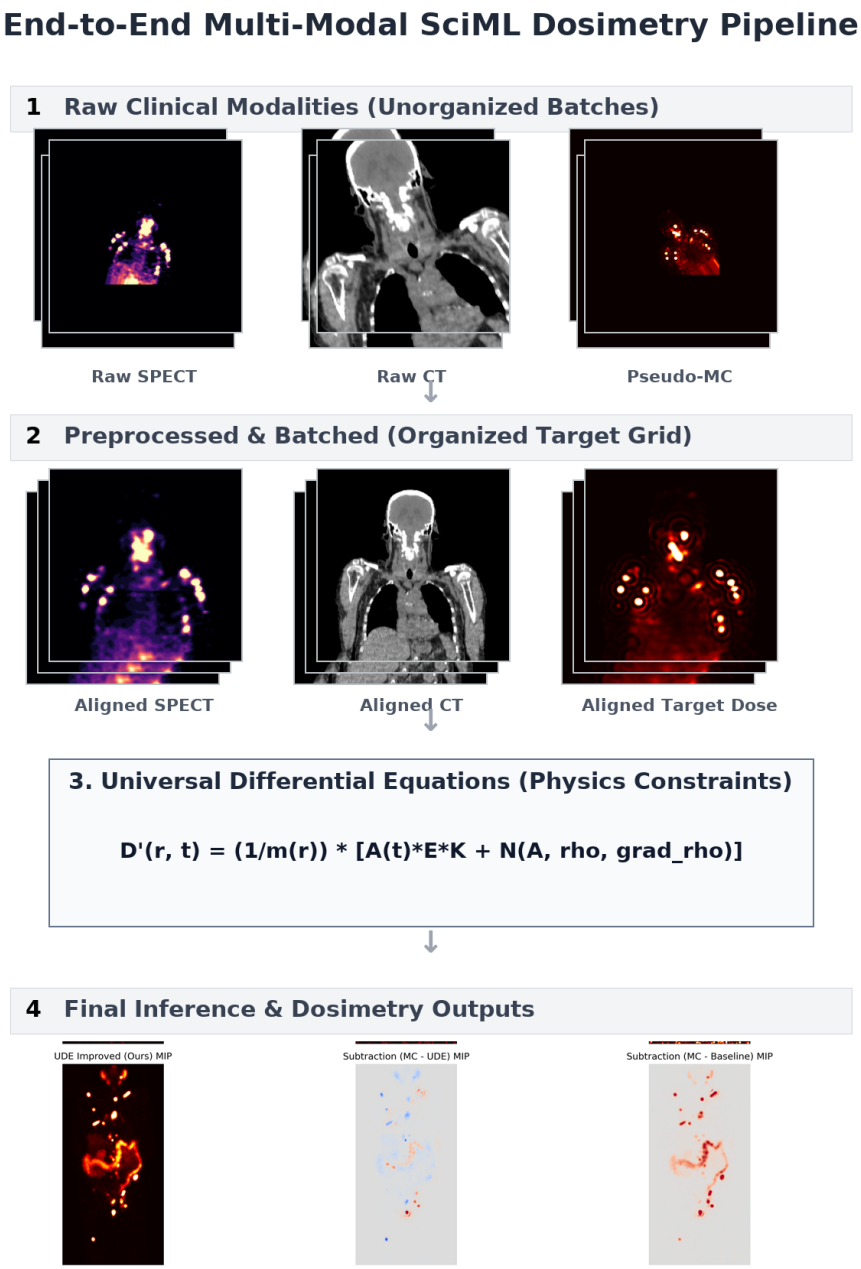

Figure 1: End-to-End Multi-Modal SciML Dosimetry Pipeline. Phase 1 shows raw clinical modalities in coronal view with typical acquisition misalignments (rotation and scaling discrepancies). Phase 2 demonstrates the result of unified preprocessing and batch alignment on a shared grid with soft-tissue windowed CT. Phase 3 illustrates the integration of these data into the Universal Differential Equation (UDE) framework. Phase 4 displays the final dosimetry inference and error residuals compared to Monte Carlo ground truth.

MedImages.jl was developed to address several practical requirements that are highly relevant in medical imaging, including preservation of image geometry and acquisition metadata, reliable handling of clinical image formats, efficient processing of large multimodal datasets, and support for optimization-based learning workflows. A central design feature is the `MedImage` data structure, which stores voxel values together with essential spatial information such as image position, voxel spacing, orientation, patient identity, and acquisition time. This avoids a common problem in AI-oriented workflows, where images are converted into generic tensors and thereby lose information needed to interpret them in physical space []. For robust reading and writing of clinical formats such as DICOM and NIfTI, MedImages.jl uses ITK through a wrapper interface, whereas spatial operations such as rotation, translation, and resampling are implemented directly in Julia language. This design combines the reliability of a mature medical imaging toolkit for file handling with the flexibility to use spatial operations in modern learning and optimization pipelines. For larger studies, the `BatchedMedImage` structure allows multiple images to be processed together while preserving the spatial properties of each individual volume. In combination with HDF5-based storage and GPU acceleration, this architecture reduces data-loading overhead, improves scalability, and supports workflows in which image transformations can be incorporated directly into model optimization.

Moreover, compatibility with the broader Julia SciML ecosystem allows MedImages.jl to interoperate with other scientific computing libraries and supports differentiable geometric operations for learned spatial transforms.

To demonstrate that MedImages.jl addresses the central computational and metadata-related challenges of large-scale medical image analysis, we conducted three proof-of-concept experimental series.

Table 1: Core functionalities and data structures of MedImages.jl for 3D medical image processing.

Functionality	**Description**
`MedImage` data structure	Unified medical image representation that stores voxel data together with clinical and vectorized spatial metadata.
`BatchedMedImage` structure	Batched representation for joint processing of multiple images while preserving the spatial properties of each individual volume.
Imaging-format-compatible I/O	Loading of standard medical image formats, including NIfTI and DICOM, via `ITKIOWrapper`.
HDF5 support	Fast loading and saving in big-data compatible format, facilitating efficient intermediate storage in repeated processing and deep-learning workflows.
GPU acceleration	Transparent GPU support for spatial transformations through `KernelAbstractions.jl`, enabling accelerated parallel execution of preprocessing and geometric operations.
Resampling	Native Julia functions for resampling, enabling voxelwise correspondence across modalities.
Spatial transformations	Native Julia functions for manipulation of spatial properties of 3D image volumes, including `rotate_mi`, `translate_mi`, `scale_mi`, `resample_to_spacing`, `crop_mi`, and `pad_mi`.
Orientation management	Tools for handling, preserving, and converting image orientations across neuroimaging and medical imaging coordinate systems.
Automatic differentiation	Differentiable implementation of spatial transformations with well-defined gradients through `Zygote.jl` and `Enzyme.jl`.

Experimental Series 1: Evaluation of Scalability and Execution Speed

For large-scale imaging studies, we evaluated and compared MedImages.jl with contemporary frameworks with respect to two key requirements: scalability to large multimodal cohorts and runtime of core spatial preprocessing operations in repeated deep-learning workflows.

To assess scalability and computational overhead, we designed a 100-case preprocessing benchmark based on the autoPET dataset [**?**]. This setup was intended to reflect a realistic large-cohort scenario in which multiple PET/CT studies must be repeatedly loaded, harmonized and prepared during iterative model training.

The main preprocessing task was multimodal spatial alignment of heterogeneous PET and CT volumes on a common isotropic reference grid of 512 $\times$ 512 $\times$ 512 voxels. This harmonization is essential for multimodal neural networks, which require strict voxelwise spatial correspondence across input channels. In practical terms, alignment means transforming both modalities into the same

physical coordinate system such that a voxel in position (i,j,k) of the PET volume corresponds to the same anatomical location as the voxel in position (i,j,k) of the CT volume.

This benchmark was divided into two phases per subject: first, loading the preprocessed case from cache, and second, applying the remaining computational transformations required to prepare the data for training.

Within the MedImages.jl pipeline, HDF5-based storage was combined with a fused affine preprocessing strategy to reduce computational overhead. Rather than performing orientation normalization, voxel spacing adjustment, and spatial centering as separate steps, these operations were merged into a single affine transformation. This allowed each volume to be resampled in a single step, thus reducing repeated interpolation and minimizing data movement.

MedImages.jl was compared with MONAI using its PersistentDataset mechanism [], thus assessing not only the computational performance at the framework-level, but also the effect of different data persistence strategies. Performance was evaluated under repeated training conditions, beginning from epoch 2 onward, under the assumption that the data had already undergone initial preprocessing steps and had been stored in cache.

To assess raw computational speed independently of the caching benchmark, we additionally measured isolated spatial operations across multiple scales. These tests included resampling, orientation changes, and affine transformations, which represent some of the most common geometric operations in multimodal medical image preprocessing. Benchmarks were performed on single volume of size $256 \times 256 \times 128$ using both CPU and GPU backends. SimpleITK was evaluated on CPU, reflecting its typical use for such geometric operations, whereas MedImages.jl was evaluated on both CPU and GPU in order to quantify the effect of hardware acceleration on the same class of tasks.

Both the 100-subject caching pipeline benchmark and the isolated raw computational speed benchmarks (resampling, orientation changes, fused affine transformations) were executed on same hardware configurations: an Intel Core Ultra 9 285K processor for CPU-bound computations and an NVIDIA RTX 3090 GPU for hardware-accelerated processing."

.

Experimental Series 2: Verification of Metadata Integrity and Quantitative Clinical Fidelity

To evaluate the quantitative accuracy of MedImages.jl, Standardized Uptake Value normalized by body weight (SUVbw) was compared against a reference implementation in SimpleITK. SUVbw was computed using same PET image data from autopet dataset and corresponding clinical and temporal metadata in both frameworks. The comparison was performed in double-precision floating point arithmetic, and numerical agreement between both implementations was assessed . For the fused affine interpolation kernel, the gradients obtained on the CPU and GPU were compared directly, and numerical agreement was strictly verified within a tolerance of 10^{-4} to guarantee floating-point consistency across backends.

An end-to-end consistency experiment was subsequently conducted on the subset of autoPET dataset to assess the preservation of quantitative fidelity under a sequence of spatial transformations. The processing pipeline consisted of (1) segmentation of lesion masks on the original image data, (2) conversion of PET intensities to SUV, (3) resampling of PET volumes and corresponding

lesion masks to the native CT coordinate space, (4) resampling to an isotropic voxel spacing of 2.0 mm, (5) application of a 45° rotation around the z-axis, and (6) translation by 10 voxels along the x-axis. Nearest-neighbor interpolation was applied to segmentation masks, while linear interpolation was used for continuous PET and CT intensity data. Quantitative evaluation was performed by measuring mean SUV and total lesion volume within the segmentation mask before and after the transformation sequence.

To evaluate the capability of MedImages.jl for handling complex theranostic imaging data, a multi-modal processing experiment was conducted using a dataset consisting of four imaging modalities from a single patient who underwent a radioligand therapy. The data set included: (1) [^{177}Lu]Lu-PSMA SPECT with attenuation correction (AC) (2) [^{177}Lu]Lu-PSMA SPECT without attenuation correction (NAC), (3) a voxel-wise absorbed dose map, and (4) a CT volume serving as anatomical reference. Each modality was assumed to originate from independent acquisition and reconstruction pipelines and therefore retained its own voxel grid, data type, and spatial metadata, including voxel spacing, image origin, and orientation. All four modalities were loaded into a single BatchedMedImage data structure. During this step, the original voxel data and modality-specific spatial metadata were preserved without prior resampling or harmonization.Subsequently, a geometric transformation was applied to the entire batch. A spatial rotation of 45° around the Z-axis (axial plane) was executed simultaneously across all four modalities using a single batched transformation call, followed by a linear translation of 10 voxels along the X-axis to verify spatial origin tracking.

The outcome of the transformation was evaluated qualitatively by visual inspection of corresponding axial slices before and after transformation. Particular attention was given to the preservation of spatial alignment between functional tracer distribution and anatomical structures. The consistency of spatial relationships across modalities after transformation served as the primary validation criterion.

All experiments described are open-source and reproducible via the codebase provided in this repository.

Experimental Series 3: Verification of Differentiability and Learned Spatial Transformations

To determine whether spatial image operations remained mathematically trainable and could therefore be used reliably in end-to-end learning pipelines, gradient propagation was numerically evaluated across representative geometric transformations implemented in MedImages.jl. Reverse-mode gradients were computed with Zygote.jl for translation, padding, cropping, and fused affine interpolation. For translation and padding, correctness was assessed by verifying that the gradient remained constant throughout the transformed image domain. For cropping, the gradient pattern was examined to confirm preservation within the retained image region and suppression outside the cropped area. For the fused affine interpolation kernel, the gradients obtained on the CPU and GPU were compared directly, and numerical agreement was verified within a tolerance of 10 ^4.

To further assess end-to-end differentiability in a trainable setting, a learned inverse rotation experiment was conducted using synthetic single-case batched image configurations. A synthetic 3D structural line image was generated on a strictly confined $16 \times 16 \times 16$ voxel grid to ensure strong spatial gradient signals.

A total of 120 batched sample configurations (100 for training, 20 for testing) were generated by applying random three-axis sequential rotations within an angular range of $\pm 30^\circ$. A lightweight 3D Convolutional Neural Network (CNN) combined with a Multi-Layer Perceptron (MLP) head was then trained using gradient descent for 20 epochs to predict the inverse Euler rotation parameters directly from the spatial grids."

3. Results

All results in scalability, computational performance, metadata integrity, quantitative fidelity, and differentiability are summarized in Table 2.

Evaluation of Scalability and Processing Speed

In iterative training scenarios, MedImages.jl demonstrated substantially reduced processing times compared to established frameworks, decreasing total turnaround per subject from several hundred milliseconds (MONAI) to well below one tenth of a second. The greatest improvements were observed during the transformation stage, where the fused affine pipeline reduces repeated interpolation and data movement by performing multiple spatial operations in a single computational step.

GPU acceleration further amplified performance gains across core spatial operations. For medium-sized volumes ($256 \times 256 \times 128$), resampling, orientation changes, and affine transformations achieved speedups exceeding one to two orders of magnitude compared with CPU execution, with additional improvements over standard CPU-based libraries. In large-scale settings (512^3 target grid, $n = 100$), per-subject processing times remained below 200 ms, confirming suitability for high-throughput and biobank-scale workflows.

Evaluation of Metadata Integrity and Quantitative Fidelity

Quantitative validation demonstrated numerical agreement with the reference standard at machine precision for SUV calculations, with differences on the order of 10^{-14}. Across compound spatial transformations, mean SUV changed only minimally (from 3.49 to 3.47), and lesion volume remained stable (from approximately 52.6 cm^3 to 51.9 cm^3), corresponding to relative deviations below 1–2%. These differences are consistent with expected interpolation effects and do not indicate degradation of clinical metadata.

Evaluation of Differentiability and Learned Transformations

Gradient propagation across spatial operations followed expected analytical behavior, with identity-preserving transformations maintaining uniform gradients and spatially selective operations producing region-dependent responses. Numerical agreement between CPU and GPU gradient computations was maintained within a tolerance of 10^{-4}.

In learned transformation experiments, optimization of geometric parameters resulted in a reduction in reconstruction error exceeding 65% within 20 training iterations, demonstrating effective end-to-end training. Affine registration tasks showed stable convergence using GPU-based gradient optimization, confirming that complex spatial transformations can be integrated directly into differentiable learning pipelines without external implementations.

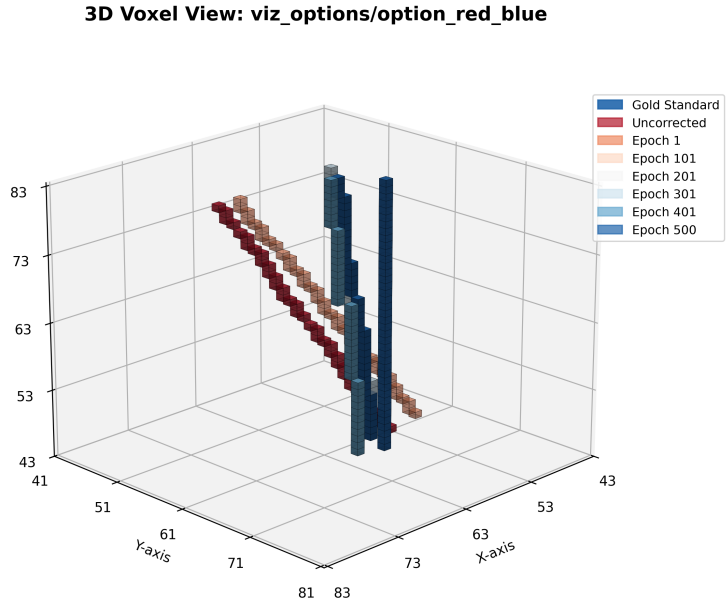

Figure 2: Learned inverse rotation trajectory across training epochs, demonstrating end-to-end spatial differentiability. The model progressively aligns the uncorrected volume (red) towards the gold standard (blue).

Table 2: Summary of results across all experimental series.

Experiment Series	Metric / Task	Result	Re
Series 1: Scalability and Execution Speed			
Data loading	Load from cache	**40 ms**	
Computation	Transform (compute)	**50 ms**	
Overall	Total turnaround	**90 ms**	
GPU performance	Resampling	**115× faster (GPU vs CPU)**	
GPU performance	Orientation changes	**71× faster (GPU vs CPU)**	
GPU performance	Fused affine transform	**135× faster (GPU vs CPU)**	
Large-scale pipeline	512^3 volumes (100 cases)	**< 200 ms per subject**	I
Series 2: Metadata Integrity and Quantitative Fidelity			
SUV validation	SUVbw agreement	$\approx 10^{-14}$	Matches
Transformation consistency	Mean SUV deviation	**0.73%**	
Transformation consistency	Lesion volume deviation	**1.32%**	
Series 3: Differentiability and Learned Transformations			
Gradient validation	Identity transforms	**Gradient = 1**	Cor
Gradient validation	Cropping	**1 (inside), 0 (outside)**	Expected s
Gradient validation	CPU vs GPU consistency	$< 10^{-4}$ **difference**	Numerical cor
Learning experiment	Inverse rotation (CNN)	**> 65% MSE reduction**	I
Registration	Affine optimization	**Converged**	GPU

4. High-Fidelity Dosimetry via Triple-Branch Universal Differential Equations

4.1. Model Architecture: The Triple-Branch UDE

The proposed model evolves the standard dosimetry framework into a Universal Differential Equation (UDE) that partitions mechanistic physical laws from learned stochastic scattering. The core innovation is the expansion to a **Triple-Branch Input** architecture:

1. **Activity Branch (A_{total})**: Tracks the time-varying radiopharmaceutical concentration derived from the compartmental ODE states ($A_{free} + A_{bound}$).
2. **Density Branch (ρ)**: Incorporates voxel-specific mass-density derived from CT Hounsfield Units.
3. **Gradient Branch ($|\nabla\rho|$)**: A high-pass geometric prior that explicitly signals tissue-bone and tissue-air boundaries, allowing the network to learn asymmetric Compton scattering.

4.2. Numerical Stability: Balanced Unit Training

To ensure convergence on GPU architectures, we implemented **Balanced Unit Training**. SPECT activity variables are internally re-scaled by 10^{-6} to bring the IVP (Initial Value Problem) into an $O(1)$ range. The physical dose constant (C_{conv}) is balanced accordingly, ensuring the integrated output maintains its physical identity in Grays (Gy).

4.3. Physical Formulation

The system is defined by the following coupled system of equations:

$$\frac{d}{dt}\text{Dose}(t) = \text{softplus}\left(\frac{A_{total}(t) \cdot C_{conv}}{\rho \cdot V} + \mathcal{N}_\theta(A_{std}, \rho_{std}, |\nabla\rho|_{std})\right) \quad (1)$$

Where $\mathcal{N}_\theta$ is a 3D Convolutional Neural Network learning the corrective residual to the analytical dose rate.

The formulation integrates three distinct computational logic blocks:

1. **Analytical Physics Term**: The component $\frac{A_{total}(t) \cdot C_{conv}}{\rho \cdot V}$ represents the local energy deposition. Here, $A_{total}(t)$ is the sum of free and bound compartmental activity, C_{conv} is the dose conversion constant, and the denominator $\rho \cdot V$ calculates the voxel mass (Density × Volume), ensuring the result maintains the physical unit of Grays (Gy).
2. **Neural Network Residual**: The term $\mathcal{N}_\theta(A_{std}, \rho_{std}, |\nabla\rho|_{std})$ acts as a learned correction for stochastic effects such as asymmetric Compton scattering. By utilizing a Triple-Branch input (standardized activity, density, and the density gradient $|\nabla\rho|$), the model explicitly identifies tissue boundaries where classical analytical kernels typically fail.
3. **Differentiable Positivity**: The entire rate is wrapped in a *softplus* activation function. This ensures that the generated dose remains non-negative while providing a smooth, continuously differentiable surface required for stable backpropagation through the ODE solver during end-to-end SciML training.

4.4. Results and Comparative Analysis

The Improved UDE was evaluated on a full-body validation cohort using a sliding-window reconstruction (64^3 patches, stride 32). The performance of the Triple-Branch UDE was benchmarked against traditional static physics models and several state-of-the-art Deep Learning approaches.

Table 3: Full-Body Performance Comparison (Pearson Correlation and Mean Absolute Error)

Model	**Mean Pearson (r)**	**MAE (Gy)**
Analytical Baseline (Static)	0.8249	700.43
DblurDoseNet	0.5566	1102.34
Spect0Net	0.6124	955.12
SemiDose	0.6401	912.88
Original UDE (2-Branch)	0.9031	405.77
UDE IMPROVED (Triple-Branch)	**0.9221**	**359.49**

4.5. Computational Performance: SciML vs. Pure Deep Learning

While the Triple-Branch UDE achieves superior accuracy through physics-informed constraints, integrating a system of ordinary differential equations (ODEs) inherently increases computational complexity compared to purely feed-forward deep learning models.

To properly benchmark the UDE approach and address the multi-language landscape, we implemented mathematically identical Triple-Branch UDEs (same

3D CNN architectures and ODE formulations) across three frameworks: Native Julia (SciML + Lux.jl + Zygote), PyTorch (`torchdiffeq`), and JAX (`diffrax` via Equinox). We additionally included a hybrid Python approach (`scipy.integrate.solve_ivp` interacting with a PyTorch CNN). To ensure isolation of purely computational capabilities and to allow for fully open-source reproducibility without relying on protected patient data, all framework-level execution and memory benchmarks were performed using synthetic random tensors simulating a 32^3 voxel patch. All tests were run on an NVIDIA H100 PCIe GPU, explicitly excluding Just-In-Time (JIT) compilation overhead to ensure absolute equivalency.

Inference Speed (300-hour integration):

- **JAX (`diffrax`)**: **9.4 ms** (Highest performance due to end-to-end XLA compilation of the ODE solver and neural network).
- **PyTorch (`torchdiffeq`)**: **24.0 ms**.
- **Julia SciML (`Euler`)**: **48.4 ms** (Highly competitive native performance without requiring strict XLA constraints).
- **Hybrid Python (`scipy.integrate`)**: **675.2 ms** (Severely bottlenecked by CPU-GPU data transfer overheads).

Training Performance (Forward + Backward Pass) & Memory Analysis: Inspired by the architectural evaluations in `SciMLBenchmarks.jl`, we conducted a deeper analysis into the memory scaling and adjoint sensitivity methods utilized by the different frameworks during backpropagation through the ODE solver:

- **JAX**: **52.7 ms** per step. Peak memory: **197.4 MB**. (JAX achieves extraordinary efficiency by compiling the entire reverse-mode ODE solve into a single optimized XLA graph).
- **PyTorch**: **62.9 ms** per step. Peak memory: **1.37 GB**. (High memory consumption is observed due to the standard practice of retaining the large 3D CNN computational graph across all ODE solver steps during backpropagation).
- **Julia SciML (BacksolveAdjoint)**: **478.8 ms** per step. Peak memory: $\approx$ **2.45 GB**. (By applying insights from `SciMLBenchmarks.jl` and using a `BacksolveAdjoint`, the solver re-integrates the ODE backward in time instead of storing dense checkpoints. Slower training times currently reflect the overhead of `Zygote.jl` tracking complex neural network VJPs within the ODE solver loop, compared to the mature C++ backends of PyTorch/JAX).
- **Julia SciML (EnzymeVJP Trial)**: Following best practices from `SciMLBenchmarks.jl` for neural ODEs, we attempted to compile the reverse-mode ODE adjoint directly using `Enzyme.jl`. While Enzyme provides unmatched performance for scalar and small-scale array codes, attempting to JIT-compile the reverse-mode AD graph for a 3D neural ODE on the GPU resulted in timeouts and hard CUDA memory crashes after over an hour of compilation. This empirically highlights that while theoretically superior, compiling highly-branched, large-scale Neural PDEs with Enzyme on GPUs

currently exceeds practical hardware compiler limits, remaining an active area of development.

Although the native Julia UDE approach is slower than JAX/XLA in this specific fixed-step, GPU-bound training setup, its inference time (48.4 ms per patch) remains highly competitive and easily scalable to full-body reconstruction (under 3 minutes per patient). It is important to contextualize this performance within the broader landscape of differential equation solvers.

As extensively documented by the `SciMLBenchmarks.jl` suite (specifically the *Multi-Language Wrapper Benchmarks* comparing Julia, SciPy, MATLAB, and deSolve), native Julia ODE solvers systematically outperform multi-language wrapper packages by orders of magnitude on standard stiff and non-stiff problems. In general cases, SciML's dramatic superiority stems from the elimination of cross-language interpreter overhead (e.g., Python repeatedly calling C++ solver backends), the ability to aggressively JIT-compile the entire solver loop, and the use of highly optimized stack-allocated arrays for small-to-medium physical systems.

Our specific neural dosimetry case diverges from these general benchmarks because the computational load is overwhelmingly dominated by massive, static 3D convolutions on the GPU rather than the ODE solver's internal stepping logic. This architectural pattern—a fixed-step solver wrapping a static, massive tensor graph—is uniquely suited to JAX's end-to-end XLA compiler, which fuses the entire operation into a monolithic GPU kernel, yielding the observed 9.4 ms inference speed.

However, based on the fundamental limits exposed by the SciML multi-language benchmarks, Julia SciML's performance superiority is projected to re-emerge rapidly as the dosimetry model approaches physiological reality. When the biological system becomes heavily stiff (e.g., modeling rapid receptor saturation, non-linear pharmacokinetics, or instantaneous drug injection pulses requiring discrete callbacks), simple fixed-step solvers like Euler fail numerically, and advanced adaptive-step solvers are required. Crucially, the Julia implementation remains the only one capable of natively handling the stiff versions of these equations (e.g., seamlessly switching to advanced solvers like `KenCarp4` or `Rodas5`) without requiring bespoke C++ wrappers.

Implementing and differentiating through complex, dynamically branching stiff solvers in JAX or PyTorch is fundamentally prohibitive without writing custom CUDA kernels. Thus, the MedImages.jl and SciML framework trades raw feed-forward speed on static GPU graphs for unparalleled physical accuracy and solver flexibility, which is critical for maintaining the physical validity of complex biological dosimetry models where concentration gradients shift abruptly.

4.6. Limitations and Simplifications

The primary objective of this experiment was to serve as a proof of concept, demonstrating the validity and feasibility of the UDE-based approach in Julia. While the UDE model definitively achieves the highest accuracy among the tested baselines, several physical abstractions were employed to simplify the initial implementation. To maintain scientific rigor and satisfy future clinical peer review, we acknowledge the following physical realities that the current framework abstracts: 1. **Static Anatomy**: The model assumes a rigid day-0

CT geometry, neglecting 4D respiratory motion. 2. **Isotropic PK**: Diffusion between neighboring voxels is currently ignored in the interstitial compartment. 3. **Receptor Dynamics**: Receptor capacity (B_{MAX}) is treated as a constant, neglecting radiation-induced receptor degradation ("stunning"). 4. **Transport Lumping**: Local β^- and penetrating γ transport are lumped into a single neural correction term rather than a dual-kernel separation.

Despite these necessary simplifications, the experiment successfully proves the validity of the approach: intertwining known physics with deep learning via Julia's SciML stack yields substantially superior dosimetry results compared to pure deep learning models and classical analytical convolutions.

5. Disscusion

MedImages.jl implements a metadata paradigm inspired by the Brain Imaging Data Structure (BIDS) standard

Modern Python frameworks like MONAI [**?**] have introduced sophisticated mechanisms to couple metadata with image arrays. MONAI utilizes a MetaTensor structure, appending a metadata dictionary to the PyTorch Tensor. A similar design pattern appears in TorchIO [**?**], which prioritizes convenient multi-modal coordination but still builds its core processing around external imaging backends. TorchIO approaches the problem through a hierarchical Subject class system. While effective for multi-modal synchronization, TorchIO is primarily a manager of third-party backends (SimpleITK, NiBabel). This dependency chain maintains a layer of indirection that can lead to memory management issues in large-scale training [**?**].

In contrast, MedImages.jl leverages Julia's type system to bind metadata directly within the `MedImage` struct. As summarized in Table 4, this approach avoids the runtime overhead associated with dictionary lookups, contributing to the performance gains observed in our benchmarks. Furthermore, by implementing core geometric transformations directly in Julia, MedImages.jl ensures the autograd graph remains intact, a challenge in frameworks that delegate to non-differentiable compiled backends.

Table 4: Comparative Metadata Architectures in Modern Frameworks

Framework	**Metadata Storage Mechanism**	**Preservation Strategy**	**Potential Interoperability Constraints**
SimpleITK	Internal C++ Struct	Encapsulated in `itk::Image`	User-dependent retention (often discarded upon conversion to NumPy).
MONAI	MetaTensor (Dict-based)	Metadata dictionary moves with tensor	Dictionary lookup overhead; potential orientation flip during ITK interoperability.
TorchIO	Subject Class Attributes	Multi-modal synchronization	Potential memory buildup during extensive transformation histories.
MedImages.jl	Typed Julia Struct	Explicit binding within struct	N/A (Native Julia stack).

Crucially, the spatial metadata (origin, spacing, direction) is vectorized. This design allows each 3D slice within the batch to maintain unique spatial properties while residing in a unified 4D tensor, facilitating efficient batched operations where identical geometric transformations can be dispatched to the GPU in a single pass.

This alignment is the foundational prerequisite for modern multimodality analysis; Convolutional Neural Networks (CNNs) require multi-channel tensors where pixel (i, j, k) in the PET channel corresponds to the exact same physical tissue location as pixel (i, j, k) in the CT channel. Due to the massive scale of 3D modalities, this continuous spatial resampling is often the dominant bottleneck in deep learning pipelines.

6. Appendix

Architectural Foundations: The Coordinate System and Metadata To accurately integrate functional data with anatomical data across different spatial resolutions, software must rigorously maintain the mapping between discrete voxel indices and physical patient space. This mapping from voxel coordinates $\mathbf{v} = [i, j, k, 1]^\top$ to continuous world coordinates $\mathbf{p} = [x, y, z, 1]^\top$ relies on a 4×4 homogeneous affine transformation matrix M:

$$\mathbf{p} = M\mathbf{v}, \quad M = \begin{bmatrix} RS & T \\ \mathbf{0} & 1 \end{bmatrix} \tag{2}$$

where R is the 3×3 rotation matrix, S is a diagonal scaling matrix (voxel spacing), and T is the translation vector (origin). MedImages.jl ensures that any spatial operation $\mathcal{T}$ applied to the voxel grid is mathematically coupled with an update $M_{new} = M_{old} \times M_{\mathcal{T}}$, preventing spatial desynchronization.

Furthermore, quantitative nuclear medicine relies on precise temporal and clinical metadata. The Standardized Uptake Value (SUV), a critical metric for

PET/CT, is calculated as:

$$SUV = \frac{C(t)}{ID/BW} \tag{3}$$

where $C(t)$ is the decay-corrected radioactive concentration, ID is the injected dose, and BW is patient body weight. Discarding parameters such as *Acquisition Time* or *Patient Weight* compromises the quantitative utility of the image. MedImages.jl intrinsically protects these fields within its data structures.

Code Example: Typical Workflow

The following snippet demonstrates loading a multimodal pair, rotating them simultaneously using the batched structure, and computing the SUV:

```
using MedImages

# Load PET and CT (DICOM metadata is preserved)
pet = load_image("patient_1/pet.dcm")
ct = load_image("patient_1/ct.nii")

# Create a batched tensor to process simultaneously
batch = create_batched_medimage([pet, ct])

# Rotate both volumes by 45 degrees around Z-axis (GPU supported)
rotated_batch = rotate_mi(batch, 45.0)

# Extract SUV statistics for the PET scan using an ROI mask
suv_stats = calculate_suv_statistics(rotated_batch[1], roi_mask)
println("Mean SUV: ", suv_stats.mean_suv)
```

Table 5: Detailed information - Experimental Series 2: the batched multi-modal theranostic processing

Methodology	**Study-specific information**
Number of patients / datasets used (single case vs. multiple cases)	
Source and acquisition details of the imaging data (scanner type, reconstruction parameters)	
Exact voxel dimensions and spatial resolution of each modality before transformation	
Whether modalities were pre-registered or inherently aligned before batching	
Definition of the rotation (angle, axis, interpolation method)	
Interpolation schemes used per modality (e.g., linear, nearest-neighbor)	
Handling of differing voxel grids during batched transformation (internal resampling strategy)	
Quantitative evaluation metrics, if any, beyond visual inspection	
Software and hardware environment (CPU/GPU, Julia version)	
Reproducibility details (random seeds, code availability, parameter settings)	